CLAIMPROMOTEBENCH: FAILURE-AWARE PAIRED TESTS OF MODEL ROUTES UNDER EXECUTABLE POLICY REVISIONS

Anonymous authors
Paper under double-blind review

ABSTRACT

Policy revisions can leave model-mediated structured decisions valid-looking but stale; valid-only scoring also hides interface failure. We introduce ClaimPromoteBench, a synthetic protocol that holds typed context fixed across executable policies and scores paired action conformance after output-contract validation. Every scheduled service, syntax, schema, and semantic failure remains incorrect end to end. In one authored `BLUE`-to-`AMBER` migration, rule-complete tests cover 6/10 action edges, graph-complete tests cover all ten reachable edges, and a 10-pair gate covers all 12 changed rules. Two pinned local routes, evaluated once without independent replication, score 10/60 and 2/60 pairs and fail the gate. Auxiliary r6 Qwen coverage is 272/300; item/pair accuracy is 258/300 and 113/150 end to end versus 258/272 and 113/123 conditional. In a balanced challenge, Qwen and Flash coverage is 148/180 and 146/180; end-to-end item/pair scores are 132/180,48/90 and 90/180,13/90, versus 132/148,48/64 and 90/146,13/61 conditional; `DEMOTE` recall is 4/30 and 0/30. A post-hoc route, frozen before its first call, returns 13/20 valid action-correct decisions and 6/10 pairs but fails the strict gate. The executor defines labels, not a learned baseline. Results support regression testing of finite authored decision surfaces, not model-family ranking, naturalistic validity, or causal comparison.

1 INTRODUCTION

Versioned policies create regression risk at model-mediated decision boundaries. The same artifact may be admitted under one profile and held under another; one failed verifier field may change an action under a fixed profile. After revision, a route can preserve stale behavior, misread evidence, or fail before producing a valid structured action. We study regression testing during maintenance or migration, not green-field enforcement. If the structured policy can replace the route, a deterministic executor is safer.

Ordinary accuracy conflates these cases. A fixed-policy dataset can reward a stable field-to-label shortcut, and evaluating only parseable responses can hide service, syntax, schema, and semantic failures. A regression suite that covers every edited rule can also miss behavioral transitions induced by the revision. We ask a bounded question: after output-contract validation, does a route emit the action prescribed by a published executable policy for a typed ledger, current state, and requested state? This is action conformance to an author-defined executable specification, not a test of exact six-field execution, scientific truth, software security, policy legitimacy, or general agent competence.

CLAIMPROMOTEBENCH treats the revision, rather than a static dataset, as the source of tests. An *evidence contrast* changes one registered field under a fixed policy. A *policy contrast* holds the complete decision context fixed and changes only the active profile. Every pair changes the prescribed action. This gives a necessary intervention-blind certificate: a route that ignores the intervention cannot pass both members. It is not sufficient evidence that a passing route followed the policy. Pair correctness requires two valid, action-correct decisions; end-to-end estimates retain every scheduled failure.

The policy contrast turns a revision into a test-generation problem. We execute both policies over a finite authored context grammar, retain their behavioral difference, and compile that set against two

adequacy targets: the changed rules witnessed by the suite and the directed action transitions it induces. Rule-complete tests can omit reachable behavior. The compiler also identifies impossible transitions and selects a cardinality-minimum joint-adequacy suite before any route output is observed.

We instantiate the protocol in two synthetic ontologies. One governs research-evidence states. The other maps SLSA and in-toto attestation fields to an authored software-artifact admission state machine (SLSA Steering Committee, 2026; in-toto Attestation Maintainers, 2026). These standards ground the field vocabulary; the authors define the policies and labels. A frozen suite can regression-test a model, prompt, decoder, adapter, or provider against the same contract without assuming a natural input distribution.

Our contributions are:

1. a synthetic protocol for paired action conformance after output-contract validation that isolates evidence or active-policy interventions, retains route failure end to end, and rejects intervention-blind shortcuts;
2. an adaptation of established semantic differencing to executable policy revisions, with separate changed-rule and observable action-transition adequacy; standard set-cover optimization, edge-local mutants, and exact dynamic programs serve as checkable implementation machinery; and
3. a fully bound evaluation of one authored migration on two pinned local route stacks, plus auxiliary frozen evidence contrasts that diagnose route failure and action imbalance without supporting model-family or causal comparisons.

2 Related Work

Behavioral and conformance testing. CheckList and contrast sets probe local decision boundaries by changing inputs under a fixed target (Ribeiro et al., 2020; Gardner et al., 2020). Metamorphic testing scores relations among transformed executions, often when a direct oracle is unavailable (Chan et al., 1998). Finite-state and ioco testing derive tests from transition specifications, and transition coverage is a standard adequacy criterion (Chow, 1978; Tretmans, 1996); DMN represents executable decision tables (Object Management Group, 2024). ClaimPromoteBench combines these ideas for a narrower setting: an exact oracle exists, the specification itself changes, and the test object is the behavioral difference between two policy versions. A survey of model-mediated software identifies regression and continuous-integration testing as under-supported in part because stable automated oracles are scarce (Gonzalez & El Mezouar, 2026).

Coverage-preserving test-suite reduction also predates our work (Harrold et al., 1993). We use that optimization view only after policy differencing has produced paired model-service tests; the adequacy target jointly covers changed rules and observable action transitions.

Semantic policy differencing is established software-engineering work. Margrave computes witnesses on which two access-control policies disagree; Cirg uses change-impact analysis to generate and reduce policy tests; mutation testing measures access-policy fault detection (Fisler et al., 2005; Martin & Xie, 2007b;a). Semantic differencing has also been applied to evolving combinatorial test models (Tzoref-Brill & Maoz, 2017). Our contribution is not the differencing operation, mutation testing, or suite reduction. We adapt them to paired model-service executions, retain output-contract failure, and distinguish changed-rule coverage from coverage of the observable four-action changes induced by a revision. The latter includes transitions between non-`ALLOW` actions that a rule-complete suite can miss.

Policy and rule reasoning. Rule-following benchmarks test conversational constraints, unusual instructions, long rule sets, policy comprehension, and request classification (Mu et al., 2024; Zhang et al., 2026; Zhou et al., 2025; Bao et al., 2026; Cisneros-Velarde, 2026). Other systems extract rule flows or internalize policy documents (Yang et al., 2026; Liu et al., 2026a). GuideBench varies domain guidelines, and augmented policy training studies unseen guardrail policies (Diao et al., 2025; Liu et al., 2026b). These works establish policy variation as prior art. They do not compile paired regression tests from the semantic difference of two executable policies or certify coverage of the induced action graph.

SpecEval synthesizes prompts from provider-authored statements and uses model judges to audit adherence; COMPASS generates queries for organization-specific allow and deny lists and judges free-form responses (Ahmed et al., 2025; Choi et al., 2026). Prompt regression instead asks whether a provider update breaks application behavior (Ma et al., 2024). We hold the route and complete typed context fixed while changing the application policy, use an exact oracle, and score both executions jointly.

Policy-to-Tests translates governance prose into executable rules, while TDAD generates agent tests, optimizes prompts, and evaluates specification evolution with semantic mutants (Datla et al., 2025; Rehan, 2026). CLAIMPROMOTEBENCH starts after that translation: it accepts two deterministic policies and a finite grammar, computes their behavioral difference, and audits changed-rule and induced-transition adequacy. It performs neither policy extraction, prompt optimization, nor judge-based labeling.

Structured output and selective prediction. Grammar-constrained decoding and incremental parsing restrict output form (Scholak et al., 2021; Geng et al., 2023); JSONSchemaBench and BFCL evaluate schemas, function selection, and arguments (Geng et al., 2025; Patil et al., 2025). Selective prediction studies risk as a model abstains (Geifman & El-Yaniv, 2017; 2019). We distinguish a valid `HOLD` action from route failure. Malformed or absent output is never treated as abstention and remains incorrect end to end.

Claim–evidence systems. ClaimBench evaluates claim–evidence reasoning in AI papers, while Evidence-Ledger Adjudication, LEDGER, and ClaimLedger organize traceable evidence or claim maintenance (Javaji et al., 2025; Chen et al., 2026; Kim et al., 2026; Elliman, 2026). CLAIMPROMOTEBENCH assumes that typed verifier facts are already registered and tests the subsequent policy-conditioned transition; it neither extracts claim–evidence links nor verifies source truth.

Research agents. ScienceAgentBench, CORE-Bench, and POPPER evaluate data analysis, computational reproduction, and sequential falsification (Chen et al., 2025; Siegel et al., 2025; Huang et al., 2025). CLAIMPROMOTEBENCH tests a smaller component that may appear inside such systems: the transition decision after evidence has already been registered. It does not evaluate evidence discovery or an end-to-end research workflow.

3 TASK AND PROTOCOL

An item is $x = (P, z, s, \tau)$, where P is the active policy, z is a typed evidence ledger, s is the current state, and τ is the requested state. The deterministic executor $\pi_P(x)$ applies first-match precedence

$$\texttt{DEMOTE} > \texttt{ESCALATE} > \texttt{HOLD} > \texttt{ALLOW}.$$

`DEMOTE` lowers a currently elevated state whose recorded basis is invalid. `ESCALATE` preserves state and opens review for an authenticated active conflict. `HOLD` rejects a requested promotion with incomplete prerequisites. `ALLOW` accepts the requested state when no preceding rule matches. The first matching row fixes the action, proposed state, and ordered reason codes.

The evaluated route receives the complete versioned policy contract, one canonical JSON ledger, and passive verifier facts. It must return one JSON object with exactly six fields: `action`, `proposed_state`, `p_admissible`, `used_evidence_ids`, `reason_codes`, and `missing_evidence_ids`. We request no rationale. Exact syntax, schema, and gold-free cross-field checks determine validity. The primary endpoint is *action conformance after output-contract validation*: it scores the prescribed action but does not require exact oracle agreement on probabilities, evidence lists, or reason codes.

Let $v(x)$ indicate a valid response and let $\hat{y}(x)$ denote its action. End-to-end item correctness is

$$C_{\text{item}}(x) = \mathbb{1}[v(x) = 1]\mathbb{1}[\hat{y}(x) = \pi_P(x)].$$

For pair $i = (x_i^a, x_i^b)$, $C_{\text{pair}}(i) = C_{\text{item}}(x_i^a)C_{\text{item}}(x_i^b)$. Decision coverage is the fraction with $v = 1$; pair coverage requires both members to be valid. Conditional accuracies divide correct items or pairs by these coverage denominators. End-to-end accuracies divide by every scheduled item or pair. Thus item accuracy equals decision coverage times conditional item accuracy, with the analogous identity

for pairs. Tables encode failure as N, not as a fifth policy action. We do not reproduce malformed response contents in the manuscript.

Intervention-blind certificate. **Proposition 1.** Let q denote the controlled coordinate: the active policy P or one registered field of z. For a pair that differs only in q and satisfies $\pi(x_i^a) \neq \pi(x_i^b)$, any deterministic route invariant to q returns the same action for both members. It can answer at most one member correctly and has $C_{\text{pair}}(i) = 0$. Across any such pair set, its end-to-end item accuracy is at most $1/2$ and its pair accuracy is zero. The policy case specializes to a classifier $h(z, s, \tau)$ that cannot observe P; the evidence case covers a route insensitive to the mutated verifier field. *Proof.* The gold actions differ while an invariant route's outputs are equal, so at most one output matches. The certificate depends on the pair construction, not model quality or training procedure. It is necessary but not sufficient: a route can react to q without executing the policy faithfully.

Policy-difference compilation. For two policies and an authored context grammar $\mathcal{X}$, let $\Delta(P_a, P_b; \mathcal{X})$ contain exactly the contexts $c = (z, s, \tau)$ for which $\pi_{P_a}(c) \neq \pi_{P_b}(c)$. Duplicating a context from Δ and changing only the active profile creates a policy-regression pair. Rule coverage records which policy differences bind in at least one pair. Behavioral coverage instead records directed edges

$$E(P_a, P_b; \mathcal{X}) = \{(\pi_{P_a}(c), \pi_{P_b}(c)) : c \in \Delta(P_a, P_b; \mathcal{X})\}.$$

Neither coverage criterion implies the other. Both are relative to the published context grammar, not to every possible deployment input.

For a test suite T, let $R(T)$ be its witnessed changed rules and $E(T)$ its induced action edges. We report migration adequacy as

$$A(T) = \left(\frac{|R(T)|}{|R_\Delta|}, \frac{|E(T)|}{|E_{\text{reach}}|} \right),$$

where R_Δ is the authored rule-difference inventory and E_{reach} is obtained by exhaustive execution over the published finite context grammar. Test construction enumerates the grammar, groups contexts by rule witness and ordered action pair, selects the registered balanced contexts, and freezes them before the evaluated route is called. Thus adequacy is established on the oracle side and cannot be chosen from model errors.

Minimum joint-adequacy suites. The compiler is independent of either ontology. Its interface accepts two deterministic policy evaluators, a finite context sequence, stable context identifiers, and authored rule witnesses; it has no SLSA, prompt, model, or provider dependency. Enumerating the behavioral difference costs $O(|\mathcal{X}|(C_a + C_b))$ for policy costs C_a, C_b. **Proposition 2.** Finding a cardinality-minimum joint rule- and edge-complete suite is NP-hard, even when every candidate has the same required edge. The reduction is given in Appendix A. For n candidates, r required rules, and k required edges, a standard exact subset dynamic program takes $O(n2^{r+k})$ state updates. Each state retains the shortest, then lexicographically first, subset for a coverage mask. An include-or-exclude induction proves optimality at the full mask. A specialized search selects one context per edge while retaining at most 2^r rule masks. If it finds a complete witness, the k-context edge lower bound proves global minimality. Both implementations reproduce the frozen witness from the published 60-pair candidate graph. These solvers are correctness-preserving implementation machinery, not an algorithmic contribution; both functions operate in one module on the same frozen input.

3.1 Two authored ontologies

The research-evidence policy uses four state axes: research status, computational support, formal certification, and independent review. Its evidence checks cover dependency closure, runtime completion, scope, freshness, provenance, identity, and contradiction status. The hidden r6 realization and public action-balanced challenge use this same policy but different pair constructions.

The software policy uses release status, accepted build level, and incident-review status. Its fields are grounded in SLSA 1.2 and in-toto Attestation 1.2, including provenance presence, signature verification, subject binding, builder and source bindings, verified build level, and authenticated conflicts (SLSA Steering Committee, 2026; in-toto Attestation Maintainers, 2026). The authored

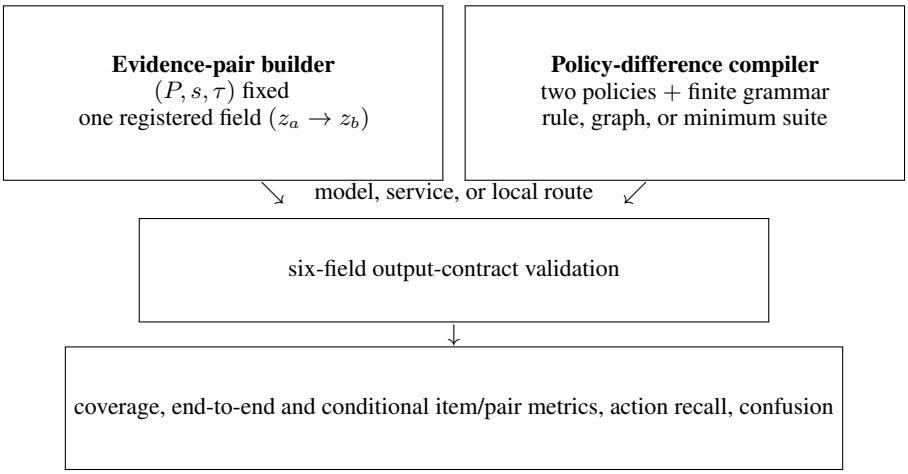


Figure 1: Two controlled paths through one failure-aware evaluator. Policy differencing also emits rule- and edge-coverage certificates, impossible edges, registered mutants, and a minimum smoke suite before route execution.

state machine maps failed candidate checks to `HOLD`, invalid current bases to `DEMOTE`, and selected authenticated conflicts to `ESCALATE`.

The policy studies publish two counterfactual profiles. `POLICY_BLUE` enforces provenance, subject, builder, and source-ref checks and escalates source-revision or VSA-policy conflicts. `POLICY_AMBER` enforces signature, attester, source-URI, and policy-digest checks and escalates build-parameter or verification-result conflicts. In the rule-stratified suite, each profile has 30 `ALLOW` decisions and ten for each other action. Both suites give the profiles identical action marginals, so a profile name alone does not predict the action. The profiles and labels are author choices; SLSA and in-toto do not endorse either one.

For example, one pair holds an admitted artifact, its evidence, and its requested state fixed while recording an unrecognized attester. `BLUE` does not bind that field and prescribes `ALLOW`; `AMBER` treats the current admission basis as invalid and prescribes `DEMOTE`. Only the profile changes.

4 Evaluation Design

Table 1 separates the central migration validation from three auxiliary diagnostics. The central evidence comprises 120 pairs from one authored policy migration: a rule-stratified suite and a transition-graph suite evaluated once on the same two pinned local routes. These are not independent replications, and the minimum gate reuses graph records. The auxiliary suites test evidence contrasts or fixed-policy output behavior; they do not validate policy differencing. R6 has rare `DEMOTE` and `ESCALATE` support. The public challenge contains 90 pairs across 15 factors and balances its 90 non-`ALLOW` members across the other three actions. The transition graph also includes pairs between non-`ALLOW` actions.

Every route receives one call per item with zero client retries. The hosted Qwen and Flash configurations were retained after an older adaptive interface study because they completed that interface; they are selected service routes, not a prospective sample of model families. The Qwen stack uses prompt-only JSON, whereas the Flash stack uses strict-schema output. Model, serving implementation, provider adapter, and output constraint therefore differ, so between-route comparisons are descriptive. Separately frozen local runs use pinned four-bit Mistral Small 3.2 24B and Phi-4 snapshots with user-only prompts, unconstrained JSON extraction, and greedy MLX-LM decoding. They cover the fixed software policy and both policy suites. Quantization, chat templates, and hosted versus local execution preclude causal comparisons.

After the preceding results were known, we selected OpenCode Go's then-listed Ox Alpha Free route (OpenCode, 2026). Before its first call, we committed the route label, exact 20-item minimum-suite schedule, zero-retry rule, complete-failure denominator, and strict 20/20-item plus 10/10-pair gate.

Table 1: Frozen evaluations. The two central rows instantiate one migration; the other rows are auxiliary diagnostics. Supports follow ALLOW/HOLD/DEMOTE/ESCALATE.

Dataset	Contrast	Policy instrument	Pairs	Items	Factors	Support
Central policy-migration validation						
Policy switch	Policy	BLUE/AMBER	60	120	12	60/20/20/20
Transition graph	Policy	BLUE/AMBER	60	120	10	36/24/24/36
Auxiliary evidence and failure diagnostics						
Research r6 (hidden)	Evidence	Research transition policy	150	300	20	150/134/8/8
Public challenge	Evidence	Research transition policy	90	180	15	90/30/30/30
SLSA portability	Evidence	Artifact-admission policy	60	120	12	60/20/20/20

This is post-hoc route selection with a pre-inference-frozen schedule. The stealth label does not identify a model family. We report the run as a time-stamped auxiliary route smoke replay, not an independent policy replication, reliability estimate, provider ranking, or causal ablation.

The research sanity control is a multinomial Naive Bayes text classifier trained on 100 public-development items; it is weak and not a competitive baseline. The fixed software policy adds a 500-tree Random Forest trained on 72 development items using flattened model-visible state and verifier fields. Each policy suite trains the same algorithm twice on its development split: the policy-aware version observes the active profile; the profile-blind version does not. Both exclude identifiers, pair metadata, evaluation labels, oracle calls, and model outputs. A profile-blind classifier receives identical features for both pair members and therefore cannot answer an action-changing pair completely. These Random Forest rows are construction and shortcut controls, not evidence of learned policy reasoning. Policy-contrast model routes were pinned before those route outputs were observed.

The deterministic executor generates every label and reproduces all items by construction. We call this the *definitional policy ceiling*. It is neither a learned baseline nor external validation of the authored policies.

The evaluation asks whether rule coverage captures the induced action graph, whether a smaller joint-adequacy gate exists, and how model-service routes behave once output-contract failures remain in scope. The first two questions are oracle-side properties; the third is empirical.

5 RESULTS

5.1 MIGRATION ADEQUACY AND MINIMUM GATE

The rule-stratified BLUE→AMBER suite covers all 12 authored profile differences with five contexts each. It contains six admission-centered transitions with ten pairs each: ALLOW↔HOLD, ALLOW↔DEMOTE, and ALLOW↔ESCALATE. The graph suite instead contains six contexts for each reachable directed edge. It covers 10/10 reachable non-self edges and all 12 rules. The remaining directions, HOLD→DEMOTE and DEMOTE→HOLD, are structurally impossible under the published executor and shared-state pair contract. DEMOTE is available only when the current state is admitted and an enforced check fails. On an admitted state, that same failure reaches DEMOTE before HOLD under first-match precedence; on a non-admitted state, DEMOTE is unavailable. Each profile has the same action support (18/12/12/18 for ALLOW/HOLD/DEMOTE/ESCALATE), blocking a profile-only class prior. The adequacy vectors are $A(T_{\text{rule}}) = (1, .6)$ and $A(T_{\text{graph}}) = (1, 1)$ on the published grammar.

We register one stale-policy mutant m_e for each reachable edge $e = (a, b)$. On an AMBER execution whose correct profile transition is e, m_e returns the stale BLUE action a. A controlled pair kills m_e if and only if it covers e. The rule-stratified suite therefore kills 6/10 registered edge mutants, whereas the graph suite kills 10/10. This oracle-side audit explains the coverage criterion; it is not a model of fault prevalence.

One pair realizes one edge, so ten reachable edges require at least ten pairs. A standard exact search over the six frozen contexts per edge finds a witness that also covers all 12 rules. This is a provably minimum 10-pair, 20-call gate for the joint $(1, 1)$ target on the published candidate pool, an 83.3% reduction from 120 calls. A digest-bound receipt publishes the selected identifiers; no model output enters selection. The search processes edge groups in canonical order and retains the lexicographically first witness for each rule mask.

On the minimum gate, Mistral produced 14/20 valid decisions, 9/20 correct items, and 2/10 correct pairs; on the minimum gate, Phi-4 produced 6/20 valid decisions, 5/20 correct items, and 0/10 correct pairs. Neither route passed the fixed 20/20-item and 10/10-pair rule. This subset reuses terminal records from the full graph run and is not an independent sample.

The later Ox Alpha route made 20 zero-retry calls on the same minimum witness. It returned 13/20 valid decisions, all action-correct, and completed six pairs, yielding 13/20 items and 6/10 pairs end to end versus 13/13 and 6/6 conditional. Three HTTP 503 responses, three cross-field semantic failures, and one length-terminated JSON response remained incorrect. The route therefore failed the strict gate despite zero incremental cash cost.

5.2 Coverage and correctness

Table 2 reports every model-route denominator. In r6, the hosted Qwen prompt-only route returned 272/300 valid decisions and achieved 258/300 item and 113/150 pair accuracy end to end.

The historical r6 Flash service interface is not a model-result row: all 300 requests returned HTTP 401 before any model output. Its decision coverage, end-to-end item score, and end-to-end pair score are therefore 0/300, 0/300, and 0/150; conditional scores are undefined. A later Flash route used a different service stack. Its success establishes only that the later path executed.

The public challenge exposes action errors among valid outputs. Its hosted Qwen prompt-only and Flash strict-schema routes achieved 132/180 and 90/180 end-to-end item accuracy, with conditional item accuracy 132/148 and 90/146. Pair accuracy fell to 48/90 and 13/90 end to end.

The fixed software policy gives a different decomposition. Every valid hosted decision is action-correct, but incomplete pairs limit end-to-end pair scores to 49/60 for Flash and 28/60 for Qwen. Local Mistral and Phi-4 add action errors after validation and score 31/60 and 23/60 pairs. On the rule-stratified policy contrast, the local routes score 38/120 and 46/120 items and 6/60 and 4/60 pairs. On the transition graph, they score 40/120 and 27/120 items and 10/60 and 2/60 pairs; conditional pair accuracy is 10/31 and 2/8. Appendix 3 separates weak and construction-dependent controls from model-route results.

5.3 Action-level diagnostics

R6 contains only eight `DEMOTE` and eight `ESCALATE` items, and the weak text sanity control recovers neither class. The public challenge raises each non-`ALLOW` support to 30: Qwen recovers 4/30 `DEMOTE` cases and Flash recovers 0/30. On the SLSA contrast, Qwen recovers 1/20 `DEMOTE` and 7/20 `ESCALATE` cases; Flash recovers 15/20 and 17/20. Both local configurations recover 0/20 `DEMOTE` cases on the fixed policy and policy switch. On the graph, both again recover 0/24. Mistral recovers 9/36 `ALLOW`, 18/24 `HOLD`, and 13/36 `ESCALATE` items; Phi-4 recovers 21/36, 5/24, and 1/36. On the Ox minimum replay, end-to-end recalls are 5/6, 4/4, 1/4, and 3/6; all 13 valid actions are correct. The small, one-context-per-edge support makes these gate diagnostics rather than class estimates. Appendix B reports every action recall and confusion cell, including no-valid-decision counts.

5.4 Factor diagnostics

Pair outcomes localize different failure modes. R6 Qwen scores 0/7 pairs on dependency-statement changes and 0/8 on current-state binding; seven of 20 factors are perfect. Challenge Qwen has three zero-accuracy factors and two perfect factors, while challenge Flash has 11 zero-accuracy factors. Each factor has only six to eight pairs, so these are diagnostics rather than stable rankings.

Table 2: Model-route coverage and correctness. E2E retains every scheduled case; conditional columns use valid decisions or complete pairs. Host/prompt denotes prompt-only JSON; host/schema denotes strict-schema output; local/4b denotes pinned four-bit MLX-LM. The final row is a separately frozen minimum-route gate.

Configuration	Valid	E2E item	Cond. item	Complete	E2E pair	Cond. pair
Research-evidence policy						
r6 Qwen (host/prompt)	272/300	258/300	258/272	123/150	113/150	113/123
Challenge Qwen (host/prompt)	148/180	132/180	132/148	64/90	48/90	48/64
Challenge Flash (host/schema)	146/180	90/180	90/146	61/90	13/90	13/61
Fixed software-artifact policy						
SLSA Qwen (host/prompt)	88/120	88/120	88/88	28/60	28/60	28/28
SLSA Flash (host/schema)	108/120	108/120	108/108	49/60	49/60	49/49
SLSA Mistral (local/4b)	94/120	91/120	91/94	34/60	31/60	31/34
SLSA Phi-4 (local/4b)	102/120	83/120	83/102	42/60	23/60	23/42
Counterfactual active-policy switch						
Switch Mistral (local/4b)	77/120	38/120	38/77	33/60	6/60	6/33
Switch Phi-4 (local/4b)	59/120	46/120	46/59	14/60	4/60	4/14
Reachable policy-transition graph						
Graph Mistral (local/4b)	71/120	40/120	40/71	31/60	10/60	10/31
Graph Phi-4 (local/4b)	43/120	27/120	27/43	8/60	2/60	2/8
Post-hoc minimum-route smoke replay						
Ox route (host/prompt; gate)	13/20	13/20	13/13	6/10	6/10	6/6

On the SLSA contrast, every covered hosted factor is correct, whereas the local routes add action errors. On the policy switch, Mistral obtains five of six correct pairs from one conflict factor; Phi-4's four correct pairs occupy three candidate-admission factors.

On the graph, Mistral's ten correct pairs occupy three edges, including all six `HOLD`→`ESCALATE` pairs; Phi-4's two correct pairs both lie on `ALLOW`→`HOLD`. Mistral has no complete pair on two edges and Phi-4 on seven. These cells diagnose the frozen routes rather than estimate edge prevalence. The artifact retains every factor numerator, denominator, failure category, and item-level scoring record.

6 Discussion

Evidence and policy contrasts test different shortcuts. The fixed SLSA Random Forest is perfect on held-out templates, but this does not establish policy conditioning. On both policy suites, removing the profile makes pair members feature-identical but label-different and yields 0/60 pair accuracy. Policy contrasts instead test whether the action changes with the published policy while evidence remains fixed.

Policy revisions yield regression tests. Rule coverage misses four reachable non-`ALLOW` edges; the graph covers all ten. The registered mutants mirror this distinction by construction, so they validate adequacy rather than estimate fault prevalence. The 20-call minimum gate does not replace the 60-pair diagnostic, which retains repeated contexts and factor localization. It rejects both local routes, but two reused subsets do not estimate the gate's sensitivity or specificity. A policy edit regenerates the difference set and certificates; a route edit reruns frozen pairs. The resulting suite can gate changes to a prompt, decoder, adapter, provider, or model: the 20-call subset gives a low-cost rejection test, while the full graph localizes failures. This pattern applies to structured admission, escalation, or routing boundaries only when executable policies and a finite grammar are available. We validate it on one authored migration. The other ontology supplies auxiliary evidence contrasts, not an independent migration replication; additional policy families remain prospective work.

Failure-aware scoring changes the diagnosis. Both SLSA hosted routes have 1.0 conditional pair accuracy, yet incomplete pairs reduce end-to-end scores to 49/60 and 28/60. Local routes add valid but wrong actions. On the balanced challenge, Qwen's 48/64 conditional pair score becomes

48/90 end to end; Flash's 13/61 becomes 13/90. Their 4/30 and 0/30 DEMOTE recalls expose a class asymmetry hidden by r6. Coverage, conditional correctness, and end-to-end correctness separate these failures. In the Ox replay, 13/13 item and 6/6 pair accuracy conditional become 13/20 and 6/10 end to end after seven failures, so the frozen gate rejects the route.

The protocol evaluates a model boundary; it does not advocate one. When policy and evidence are fully structured, a deterministic executor should enforce the transition. The routes represent an existing boundary under contract change or migration. CLAIMPROMOTEBENCH supplies its regression oracle; it does not show that a model is necessary or preferable.

Route differences are descriptive. The old r6 Flash interface produced 300 HTTP 401 responses. Later Flash configurations changed the adapter and output contract; this proves neither model improvement nor an ablation effect. Hosted and local routes also differ in weights, quantization, prompts, adapters, and constraints.

7 LIMITATIONS

All labels come from authored synthetic policies. Compilation assumes two deterministic policies, authored rule witnesses, and a finite grammar; it neither extracts rules from prose nor exhausts deployment behavior. Action edges are recomputed, but incorrect witnesses would invalidate the rule-coverage certificate. SLSA and in-toto ground the software fields but do not endorse our state machine. The central result uses one migration, one grammar, two local snapshots on one stack, greedy one-shot decoding, and no independent replication. Two ontologies do not define a policy population, and the profiles are fixed instruments rather than unseen-policy synthesis.

R6 and the public challenge share one ontology and mostly the same templates; the challenge is public robustness evidence, not hidden-test generalization. Canonical ledgers leave extraction, ambiguity, policy retrieval, and downstream outcomes untested. Reviewing 12 source-traceable CORE-Bench candidates created zero cases and zero model calls because time-stamped artifacts and provenance were missing. Naturalistic validity therefore remains untested. No human annotators or agreement statistic are claimed.

Each configuration receives one call per item with zero retries. The results do not estimate sampling variance, temporal stability, retry policies, or a model population. The primary endpoint is action conformance after output-contract validation, not six-field exact match or evidence of policy understanding. Local routes use four-bit snapshots on one Apple Silicon stack; route differences are not causal.

The Ox route was selected after other outcomes were known but scheduled before its first call. Its stealth label does not identify a model family. Limited-time service status, one context per edge, and one pass preclude claims about replication, temporal reliability, weights, or provider effects.

Naive Bayes is weak; the Random Forest controls expose direct feature shortcuts rather than competitive reasoning. Profile ablation closes one shortcut but does not establish rule following under paraphrase, long policies, adversarial evidence, or unseen composition.

Edge-local mutants cover only their registered family; optimality is candidate-pool-relative, and the gate reuses graph records. Quota-confounded hosted policy-switch studies are excluded in full; no successful hosted row or conditional subset is selected. A separate 40-cell hosted smoke is excluded in full; no row from that run enters the paper. Invalid response contents are omitted while their counts remain in end-to-end denominators and auditable records.

8 CONCLUSION

CLAIMPROMOTEBENCH compiles one executable policy revision into paired, failure-aware model-route tests. Rule coverage misses four of ten reachable action edges; an exact search retains all 12 changed rules in a 20-call gate. The two pinned local routes score 10/60 and 2/60 pairs; Ox fails the same gate despite perfect valid-only action scores. Reporting coverage, action conformance, and end-to-end correctness supports regression tests of finite authored policy surfaces without establishing model-family quality, naturalistic validity, exact policy understanding, or causal effects.

9 REPRODUCIBILITY STATEMENT

The artifact supports metric and contract replay. It binds released policies, datasets, prompts, terminal outcome classes, analyses, and privacy receipts. Validators recompute every reported metric and confusion cell from item records. The anonymous derivative removes raw response text, free-form errors, provider identifiers, hidden reasoning, account identity, and private commit identity; its manifest derives file counts from the generated tree. The Ox derivative retains all 20 item-level action and failure classes while omitting full structured outputs. Hosted services are time-dependent, local weights remain external, and exact hardware-specific inference is not reproduced. Completed one-shot studies are never rerun.

10 AI ASSISTANCE DISCLOSURE

Generative AI tools assisted literature search, planning, code and benchmark drafting, debugging, analysis checks, and manuscript editing. The author chose the research question, policies, protocols, exclusions, analyses, and claims; verified the datasets, results, artifacts, and references; and is responsible for the manuscript. AI systems are neither authors nor human annotators.

REFERENCES

Ahmed Ahmed, Kevin Klyman, Yi Zeng, Sanmi Koyejo, and Percy Liang. SpecEval: Evaluating model adherence to behavior specifications, 2025. URL https://arxiv.org/abs/2509.02464.

Han Bao, Penghao Zhang, Yue Huang, Zhengqing Yuan, Yanchi Ru, Su Rui, Yujun Zhou, Xiangqi Wang, Kehan Guo, Nitesh V. Chawla, Yanfang Ye, and Xiangliang Zhang. PolicyLLM: Towards excellent comprehension of public policy for large language models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 2249–2274. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.107. URL https://aclanthology.org/2026.findings-acl.107/.

F. T. Chan, T. Y. Chen, Shing Chi Cheung, M. F. Lau, and S. M. Yiu. Application of metamorphic testing in numerical analysis. In *Proceedings of the IASTED International Conference on Software Engineering*, 1998. URL https://hdl.handle.net/1783.1/70576.

Gengyu Chen, Yongjie Yu, and Weiling Wang. Evidence-ledger adjudication for claim-evidence traceability. *arXiv preprint arXiv:2607.26512*, 2026. URL https://arxiv.org/abs/2607.26512.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=6z4YKr0GK6.

Dasol Choi, DongGeon Lee, Brigitta Jesica Kartono, Helena Berndt, Taeyoun Kwon, Joonwon Jang, Haon Park, Hwanjo Yu, and Minsuk Kahng. COMPASS: A framework for evaluating organization-specific policy alignment in LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 46087–46133. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.2139. URL https://aclanthology.org/2026.acl-long.2139/.

Tsun S. Chow. Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering*, SE-4(3):178–187, 1978. doi: 10.1109/TSE.1978.231496. URL https://doi.org/10.1109/TSE.1978.231496.

Pedro Cisneros-Velarde. Policy compliance of user requests in natural language for AI systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 318–328. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-industry.21. URL https://aclanthology.org/2026.acl-industry.21/.

Gautam Varma Datla, Anudeep Vurity, Tejaswani Dash, Tazeem Ahmad, Mohd Adnan, and Saima Rafi. Executable governance for AI: Translating policies into rules using LLMs, 2025. URL https://arxiv.org/abs/2512.04408. AAAI 2026 AI Governance Workshop.

Lingxiao Diao, Xinyue Xu, Wanxuan Sun, Cheng Yang, and Zhuosheng Zhang. GuideBench: Benchmarking domain-oriented guideline following for LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11361–11399. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.557. URL https://aclanthology.org/2025.acl-long.557/.

David Elliman. ClaimLedger: Verifier-governed claim maintenance for AI-assisted research. Technical report, Neuro-Symbolic Ltd., 2026. URL https://neusym.ai/papers/claimledger.pdf. Contemporaneous technical report; accessed August 2026.

Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. Verification and change-impact analysis of access-control policies. In *Proceedings of the 27th International Conference on Software Engineering*, pp. 196–205. ACM, 2005. doi: 10.1145/1062455.1062502. URL https://doi.org/10.1145/1062455.1062502.

Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco, Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, Sameer Singh, Noah A. Smith, Sanjay Subramanian, Reut Tsarfaty, Eric Wallace, Ally Zhang, and Ben Zhou. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1307–1323. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.117. URL https://aclanthology.org/2020.findings-emnlp.117/.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Yonatan Geifman and Ran El-Yaniv. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2151–2159. PMLR, 2019. URL https://proceedings.mlr.press/v97/geifman19a.html.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10932–10952. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.674. URL https://aclanthology.org/2023.emnlp-main.674/.

Saibo Geng, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*, 2025. doi: 10.48550/arXiv.2501.10868. URL https://arxiv.org/abs/2501.10868.

Karla Gonzalez and Mariam El Mezouar. Testing AIware systems: A software engineering survey. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*, pp. 71–86. Association for Computing Machinery, 2026. doi: 10.1145/3805760.3814894. URL https://doi.org/10.1145/3805760.3814894.

Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology*, 2(3):270–285, 1993. doi: 10.1145/152388.152391. URL https://doi.org/10.1145/152388.152391.

Kexin Huang, Ying Jin, Ryan Li, Michael Y. Li, Emmanuel Candes, and Jure Leskovec. Automated hypothesis validation with agentic sequential falsifications. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 25372–25437. PMLR, 2025. URL https://proceedings.mlr.press/v267/huang25n.html.

in-toto Attestation Maintainers. *in-toto Attestation Framework Specification, Version 1.2*. Cloud Native Computing Foundation, 2026. URL https://github.com/in-toto/attestation/tree/v1.2.0.

Shashidhar Reddy Javaji, Yupeng Cao, Haohang Li, Yangyang Yu, Nikhil Muralidhar, and Zining Zhu. Can AI validate science? benchmarking LLMs on claim–evidence reasoning in AI papers. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 2355–2379, 2025. doi: 10.18653/v1/2025.ijcnlp-long.127. URL https://aclanthology.org/2025.ijcnlp-long.127/.

Daehong Kim, Haichao Miao, and Shusen Liu. LEDGER: Claim-to-evidence trace graphs for auditing LLM agents. *arXiv preprint arXiv:2608.18398*, 2026. URL https://arxiv.org/abs/2608.18398.

Jiateng Liu, Zhenhailong Wang, Xiaojiang Huang, Yingjie Li, Xiang Li, Chenlei Guo, Xing Fan, Ruhi Sarikaya, and Heng Ji. Analyzing and internalizing complex policy documents for LLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16814–16851. Association for Computational Linguistics, 2026a. doi: 10.18653/v1/2026.acl-long.767. URL https://aclanthology.org/2026.acl-long.767/.

Minqian Liu, Ioana Baldini, David Rabinowitz, David S. Rosenberg, Sebastian Gehrmann, and Mark Dredze. Domain generalizable AI guardrails with augmented policy training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16452–16469. Association for Computational Linguistics, 2026b. doi: 10.18653/v1/2026.acl-long.748. URL https://aclanthology.org/2026.acl-long.748/.

Wanqin Ma, Chenyang Yang, and Christian Kästner. (why) is my prompt getting worse? rethinking regression testing for evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering–Software Engineering for AI*, pp. 166–171. Association for Computing Machinery, 2024. doi: 10.1145/3644815.3644950. URL https://doi.org/10.1145/3644815.3644950.

Evan Martin and Tao Xie. A fault model and mutation testing of access control policies. In *Proceedings of the 16th International Conference on World Wide Web*, pp. 667–676. ACM, 2007a. doi: 10.1145/1242572.1242663. URL https://doi.org/10.1145/1242572.1242663.

Evan Martin and Tao Xie. Automated test generation for access control policies via change-impact analysis. In *Third International Workshop on Software Engineering for Secure Systems*, pp. 5. IEEE Computer Society, 2007b. doi: 10.1109/SESS.2007.5. URL https://doi.org/10.1109/SESS.2007.5.

Norman Mu, Sarah Chen, Zifan Wang, Sizhe Chen, David Karamardian, Lulwa Aljeraisy, Basel Alomair, Dan Hendrycks, and David Wagner. Can LLMs follow simple rules?, 2024. URL https://arxiv.org/abs/2311.04235.

Object Management Group. *Decision Model and Notation (DMN), Version 1.5*. Object Management Group, 2024. URL https://www.omg.org/spec/DMN/1.5/PDF.

OpenCode. OpenCode Go. Official service documentation, 2026. URL https://opencode.ai/docs/go/. Accessed 2026-08-24.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Tzafrir Rehan. Test-driven AI agent definition (TDAD): Compiling tool-using agents from behavioral specifications, 2026. URL https://arxiv.org/abs/2603.08806.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.779. URL https://aclanthology.org/2021.emnlp-main.779/.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=BsMMc4MEGS.

SLSA Steering Committee. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2*. Open Source Security Foundation, 2026. URL https://slsa.dev/spec/v1.2/. Build provenance, artifact verification, and verification summary specifications.

Jan Tretmans. Conformance testing with labelled transition systems: Implementation relations and test generation. *Computer Networks and ISDN Systems*, 29(1):49–79, 1996. doi: 10.1016/S0169-7552(96)00017-7. URL https://doi.org/10.1016/S0169-7552(96)00017-7.

Rachel Tzoref-Brill and Shahar Maoz. Syntactic and semantic differencing for combinatorial models of test designs. In *Proceedings of the 39th International Conference on Software Engineering*, pp. 621–631. IEEE/ACM, 2017. doi: 10.1109/ICSE.2017.63. URL https://doi.org/10.1109/ICSE.2017.63.

Chen Yang, Ruping Xu, Ruizhe Li, Bin Cao, and Jing Fan. Business as rulesual: A benchmark and framework for business rule flow modeling with LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 35198–35220. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1625. URL https://aclanthology.org/2026.acl-long.1625/.

Qinyan Zhang, Xinping Lei, Ruijie Miao, Fu Yu, Haojie Fan, Le Chang, Jiafan Hou, Dingling Zhang, Zhongfei Hou, Ziqiang Yang, Changxin Pu, Fei Hu, Jingkai Liu, Jiaheng Liu, Tong Yang, Zaiyuan Wang, Ge Zhang, Xinjie Chen, Jianpeng Jiao, and Wenhao Huang. Inverse IFEval: Can LLMs unlearn stubborn training conventions to follow real instructions? In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d1a1e8713fcd5626656553c82f7c3b26-Abstract-Conference.html.

Ruiwen Zhou, Wenyue Hua, Liangming Pan, Sitao Cheng, Xiaobao Wu, En Yu, and William Yang Wang. RuleArena: A benchmark for rule-guided reasoning with LLMs in real-world scenarios. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 550–572. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.27. URL https://aclanthology.org/2025.acl-long.27/.

A Compiler Guarantees

Proof of Proposition 2. Take a Set Cover instance with universe U and subsets $S_1, \ldots, S_n$. Create one required rule for each element of U and one candidate context for each S_i whose rule-witness set is S_i. Give every candidate the same required action edge. Any nonempty selection covers that edge, and it covers all required rules exactly when the corresponding subsets cover U. A minimum joint-adequacy suite therefore gives a minimum set cover. The construction is polynomial.

Unreachable HOLD/DEMOTE directions. Policy-contrast members share current state. Under the published first-match executor, `DEMOTE` is available only for an admitted state with a failed enforced check. For an admitted state, that failure selects `DEMOTE` before the executor can reach `HOLD`. For any non-admitted state, `DEMOTE` is unavailable. No shared state can therefore support either `HOLD`→`DEMOTE` or `DEMOTE`→`HOLD`. This argument supersedes the frozen receipt's narrower statement that `HOLD` requires a candidate state; `HOLD` can also occur for a quarantined state. A digest-bound post-freeze interpretation addendum corrects the premise without changing the edge set, labels, coverage counts, selected suite, or route results.

Exact dynamic program. Index the $r + k$ required rules and edges and encode coverage as a bit mask. After processing the first j candidates, the table stores, for every reachable mask, the shortest subset that realizes that mask, breaking ties by context identifier. This invariant holds initially for the empty mask. For candidate $j + 1$, the exclude transition preserves every prior state and the include transition unions its coverage mask with each prior mask. Keeping the shorter, then lexicographically first, representative preserves the invariant. At $j = n$, the representative for the full mask is therefore a cardinality-minimum joint-complete suite. This standard bit-mask implementation performs $O(n2^{r+k})$ state updates and stores at most 2^{r+k} states.

Edge lower-bound certificate. Each policy-contrast pair realizes one directed action edge. Covering k distinct required edges therefore needs at least k pairs. The specialized search chooses one candidate from each edge group while tracking rule masks. If its full-rule state exists, the returned k-pair suite meets the lower bound and is globally minimum. The generic and specialized functions both reproduce the frozen ten-pair witness from the same input. The exact implementation also matches exhaustive subset enumeration on 64 deterministic small candidate pools.

B Complete Action-Level Results

Table 3: Sanity and shortcut controls, separated from model-route results. These rows check task construction or direct feature shortcuts; they are not competitive learned baselines.

Control	Valid	E2E item	Cond. item	Complete	E2E pair	Cond. pair
r6 NB sanity	300/300	184/300	184/300	150/150	34/150	34/150
SLSA RF sanity	120/120	120/120	120/120	60/60	60/60	60/60
Switch RF/profile	120/120	120/120	120/120	60/60	60/60	60/60
Switch RF/blind	120/120	60/120	60/120	60/60	0/60	0/60
Graph RF/profile	120/120	107/120	107/120	60/60	47/60	47/60
Graph RF/blind	120/120	57/120	57/120	60/60	0/60	0/60

Table 4: End-to-end recall by gold action. Every no-valid-decision cell is a miss. Bal. is the unweighted mean over the four recalls.

Configuration	ALLOW	HOLD	DEMOTE	ESCALATE	Bal.
r6 Qwen (host/prompt)	.9533	.8209	.0000	.6250	.5998
r6 NB sanity	.5000	.8134	.0000	.0000	.3284
Challenge Qwen (host/prompt)	.8778	.8667	.1333	.7667	.6611
Challenge Flash (host/schema)	.7556	.3000	.0000	.4333	.3722
SLSA Qwen (host/prompt)	1.0000	1.0000	.0500	.3500	.6000
SLSA Flash (host/schema)	.9833	.8500	.7500	.8500	.8583
SLSA Mistral (local/4b)	1.0000	.9500	.0000	.6000	.6375
SLSA Phi-4 (local/4b)	1.0000	.2000	.0000	.9500	.5375
SLSA RF sanity	1.0000	1.0000	1.0000	1.0000	1.0000
Switch Mistral (local/4b)	.2667	.6000	.0000	.5000	.3417
Switch Phi-4 (local/4b)	.6333	.3000	.0000	.1000	.2583
Switch RF/profile	1.0000	1.0000	1.0000	1.0000	1.0000
Switch RF/blind	.0000	1.0000	1.0000	1.0000	.7500
Graph Mistral (local/4b)	.2500	.7500	.0000	.3611	.3403
Graph Phi-4 (local/4b)	.5833	.2083	.0000	.0278	.2049
Graph RF/profile	.8889	.7917	1.0000	.8889	.8924
Graph RF/blind	.2500	.5833	1.0000	.2778	.5278
Ox route (host/prompt; gate)	.8333	1.0000	.2500	.5000	.6458

Table 5: Conditional recall by gold action among valid decisions. N/E denotes no valid decision for that gold action. This table must be read with the end-to-end recalls above.

Configuration	ALLOW	HOLD	DEMOTE	ESCALATE
r6 Qwen (host/prompt)	1.0000	.9483	.0000	1.0000
r6 NB sanity	.5000	.8134	.0000	.0000
Challenge Qwen (host/prompt)	1.0000	1.0000	.2000	1.0000
Challenge Flash (host/schema)	.8500	.4091	.0000	.8125
SLSA Qwen (host/prompt)	1.0000	1.0000	1.0000	1.0000
SLSA Flash (host/schema)	1.0000	1.0000	1.0000	1.0000
SLSA Mistral (local/4b)	1.0000	.9500	.0000	1.0000
SLSA Phi-4 (local/4b)	1.0000	.2222	.0000	.9500
SLSA RF sanity	1.0000	1.0000	1.0000	1.0000
Switch Mistral (local/4b)	.3636	.6000	N/E	.7692
Switch Phi-4 (local/4b)	1.0000	1.0000	.0000	1.0000
Switch RF/profile	1.0000	1.0000	1.0000	1.0000
Switch RF/blind	.0000	1.0000	1.0000	1.0000
Graph Mistral (local/4b)	.3462	.7500	.0000	.7222
Graph Phi-4 (local/4b)	1.0000	.8333	.0000	1.0000
Graph RF/profile	.8889	.7917	1.0000	.8889
Graph RF/blind	.2500	.5833	1.0000	.2778
Ox route (host/prompt; gate)	1.0000	1.0000	1.0000	1.0000

The following tables give every end-to-end confusion cell explicitly. N means no valid decision and is not a policy action.

Table 6: Research-evidence route and sanity-control confusion counts. Columns are predicted actions.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
r6 Qwen (host/prompt)	ALLOW	143	0	0	0	7
r6 Qwen (host/prompt)	HOLD	4	110	0	2	18
r6 Qwen (host/prompt)	DEMOTE	8	0	0	0	0
r6 Qwen (host/prompt)	ESCALATE	0	0	0	5	3
r6 NB sanity	ALLOW	75	75	0	0	0
r6 NB sanity	HOLD	25	109	0	0	0
r6 NB sanity	DEMOTE	8	0	0	0	0
r6 NB sanity	ESCALATE	8	0	0	0	0
Challenge Qwen (host/prompt)	ALLOW	79	0	0	0	11
Challenge Qwen (host/prompt)	HOLD	0	26	0	0	4
Challenge Qwen (host/prompt)	DEMOTE	8	4	4	4	10
Challenge Qwen (host/prompt)	ESCALATE	0	0	0	23	7
Challenge Flash (host/schema)	ALLOW	68	4	0	8	10
Challenge Flash (host/schema)	HOLD	13	9	0	0	8
Challenge Flash (host/schema)	DEMOTE	25	3	0	0	2
Challenge Flash (host/schema)	ESCALATE	0	3	0	13	14

Table 7: Fixed software-policy confusion counts. Columns are predicted actions.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
SLSA Qwen (host/prompt)	ALLOW	60	0	0	0	0
SLSA Qwen (host/prompt)	HOLD	0	20	0	0	0
SLSA Qwen (host/prompt)	DEMOTE	0	0	1	0	19
SLSA Qwen (host/prompt)	ESCALATE	0	0	0	7	13
SLSA Flash (host/schema)	ALLOW	59	0	0	0	1
SLSA Flash (host/schema)	HOLD	0	17	0	0	3
SLSA Flash (host/schema)	DEMOTE	0	0	15	0	5
SLSA Flash (host/schema)	ESCALATE	0	0	0	17	3
SLSA Mistral (local/4b)	ALLOW	60	0	0	0	0
SLSA Mistral (local/4b)	HOLD	1	19	0	0	0
SLSA Mistral (local/4b)	DEMOTE	0	2	0	0	18
SLSA Mistral (local/4b)	ESCALATE	0	0	0	12	8
SLSA Phi-4 (local/4b)	ALLOW	60	0	0	0	0
SLSA Phi-4 (local/4b)	HOLD	14	4	0	0	2
SLSA Phi-4 (local/4b)	DEMOTE	4	0	0	0	16
SLSA Phi-4 (local/4b)	ESCALATE	1	0	0	19	0
SLSA RF sanity	ALLOW	60	0	0	0	0
SLSA RF sanity	HOLD	0	20	0	0	0
SLSA RF sanity	DEMOTE	0	0	20	0	0
SLSA RF sanity	ESCALATE	0	0	0	20	0

Table 8: Rule-stratified policy-switch confusion counts. Columns are predicted actions.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
Switch Mistral (local/4b)	ALLOW	16	13	0	15	16
Switch Mistral (local/4b)	HOLD	8	12	0	0	0
Switch Mistral (local/4b)	DEMOTE	0	0	0	0	20
Switch Mistral (local/4b)	ESCALATE	3	0	0	10	7
Switch Phi-4 (local/4b)	ALLOW	38	0	0	0	22
Switch Phi-4 (local/4b)	HOLD	0	6	0	0	14
Switch Phi-4 (local/4b)	DEMOTE	4	9	0	0	7
Switch Phi-4 (local/4b)	ESCALATE	0	0	0	2	18
Switch RF/profile	ALLOW	60	0	0	0	0
Switch RF/profile	HOLD	0	20	0	0	0
Switch RF/profile	DEMOTE	0	0	20	0	0
Switch RF/profile	ESCALATE	0	0	0	20	0
Switch RF/blind	ALLOW	0	20	20	20	0
Switch RF/blind	HOLD	0	20	0	0	0
Switch RF/blind	DEMOTE	0	0	20	0	0
Switch RF/blind	ESCALATE	0	0	0	20	0

Table 9: Reachable policy-transition graph confusion counts. Columns are predicted actions.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
Graph Mistral (local/4b)	ALLOW	9	8	0	9	10
Graph Mistral (local/4b)	HOLD	2	18	0	4	0
Graph Mistral (local/4b)	DEMOTE	3	0	0	0	21
Graph Mistral (local/4b)	ESCALATE	2	3	0	13	18
Graph Phi-4 (local/4b)	ALLOW	21	0	0	0	15
Graph Phi-4 (local/4b)	HOLD	1	5	0	0	18
Graph Phi-4 (local/4b)	DEMOTE	3	12	0	0	9
Graph Phi-4 (local/4b)	ESCALATE	0	0	0	1	35
Ox route (host/prompt; gate)	ALLOW	5	0	0	0	1
Ox route (host/prompt; gate)	HOLD	0	4	0	0	0
Ox route (host/prompt; gate)	DEMOTE	0	0	1	0	3
Ox route (host/prompt; gate)	ESCALATE	0	0	0	3	3
Graph RF/profile	ALLOW	32	0	3	1	0
Graph RF/profile	HOLD	5	19	0	0	0
Graph RF/profile	DEMOTE	0	0	24	0	0
Graph RF/profile	ESCALATE	3	0	1	32	0
Graph RF/blind	ALLOW	9	8	12	7	0
Graph RF/blind	HOLD	7	14	0	3	0
Graph RF/blind	DEMOTE	0	0	24	0	0
Graph RF/blind	ESCALATE	8	6	12	10	0